

คู่มือฉบับสมบูรณ์สู่โลกของ Docker

จากแนวคิดเริ่มต้น สู่การใช้งานระดับองค์กร

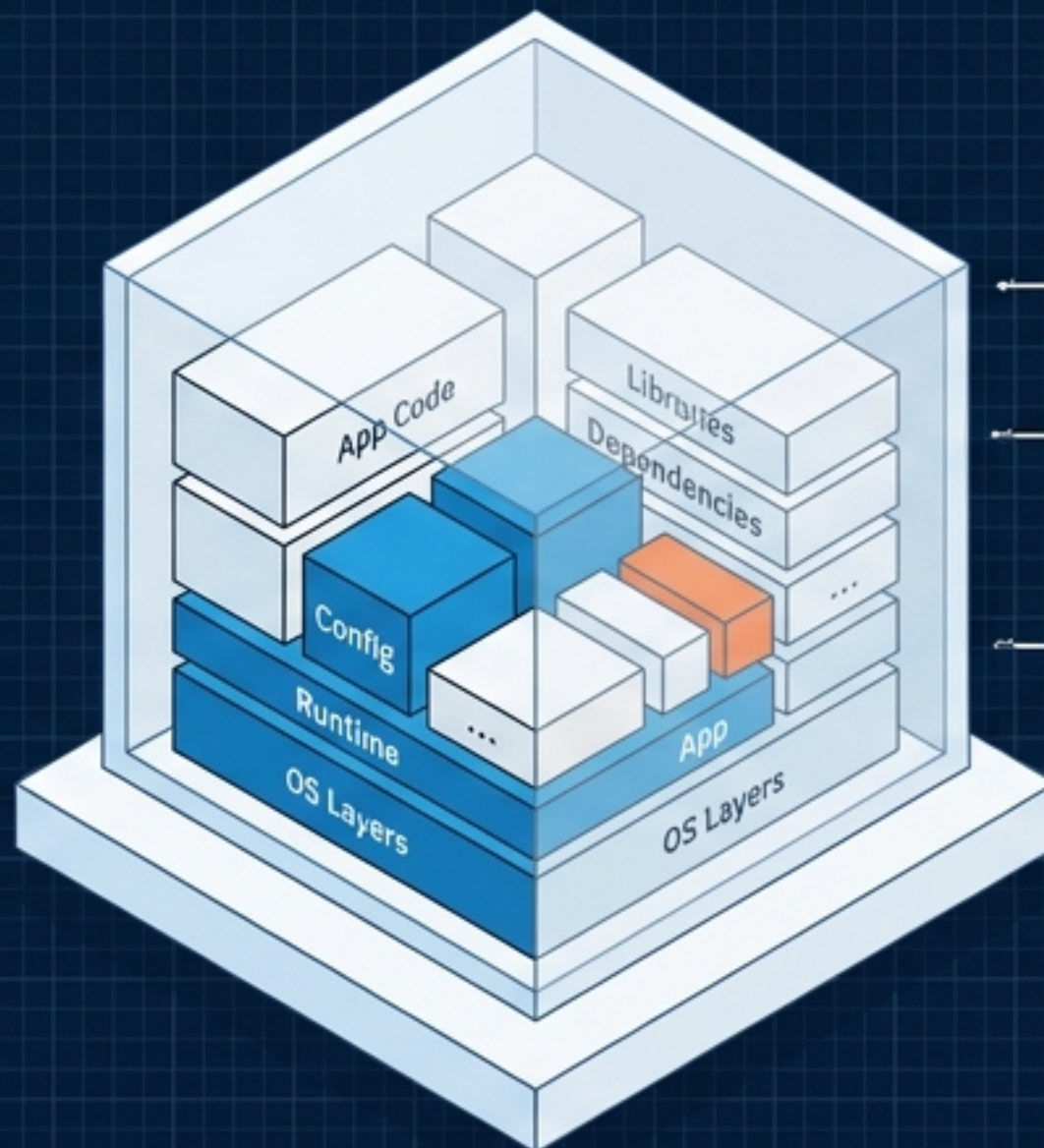


AskMe

ตัวแทนจำหน่าย Docker
อย่างเป็นทางการรายแรกในประเทศไทย

บอกเล่าปัญหา "รันบนเครื่องฉันได้ แต่บนเซิร์ฟเวอร์พัง"

Docker คือแพลตฟอร์มที่รวมแอปพลิเคชันและสภาพแวดล้อมที่จำเป็นทั้งหมดไว้ใน "Container" เดียว ทำให้ทำงานได้อย่างสมบูรณ์แบบไม่ว่าจะย้ายไปรันบนแพลตฟอร์มใด



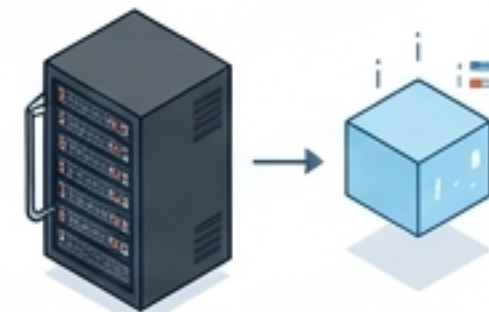
Isolated (แยกส่วนอย่างอิสระ)

ไม่กวนระบบหลัก หรือแอปพลิเคชันอื่น



Lightweight (น้ำหนักเบา)

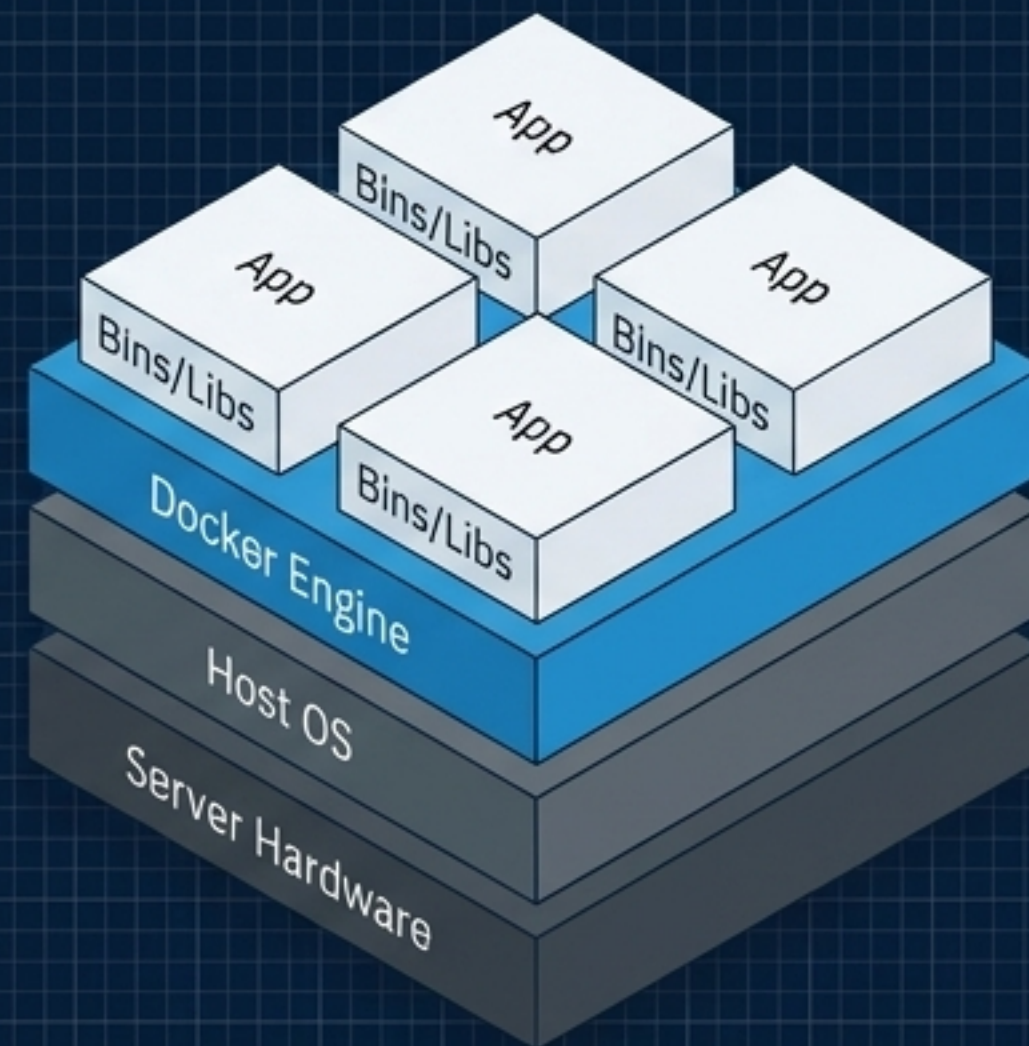
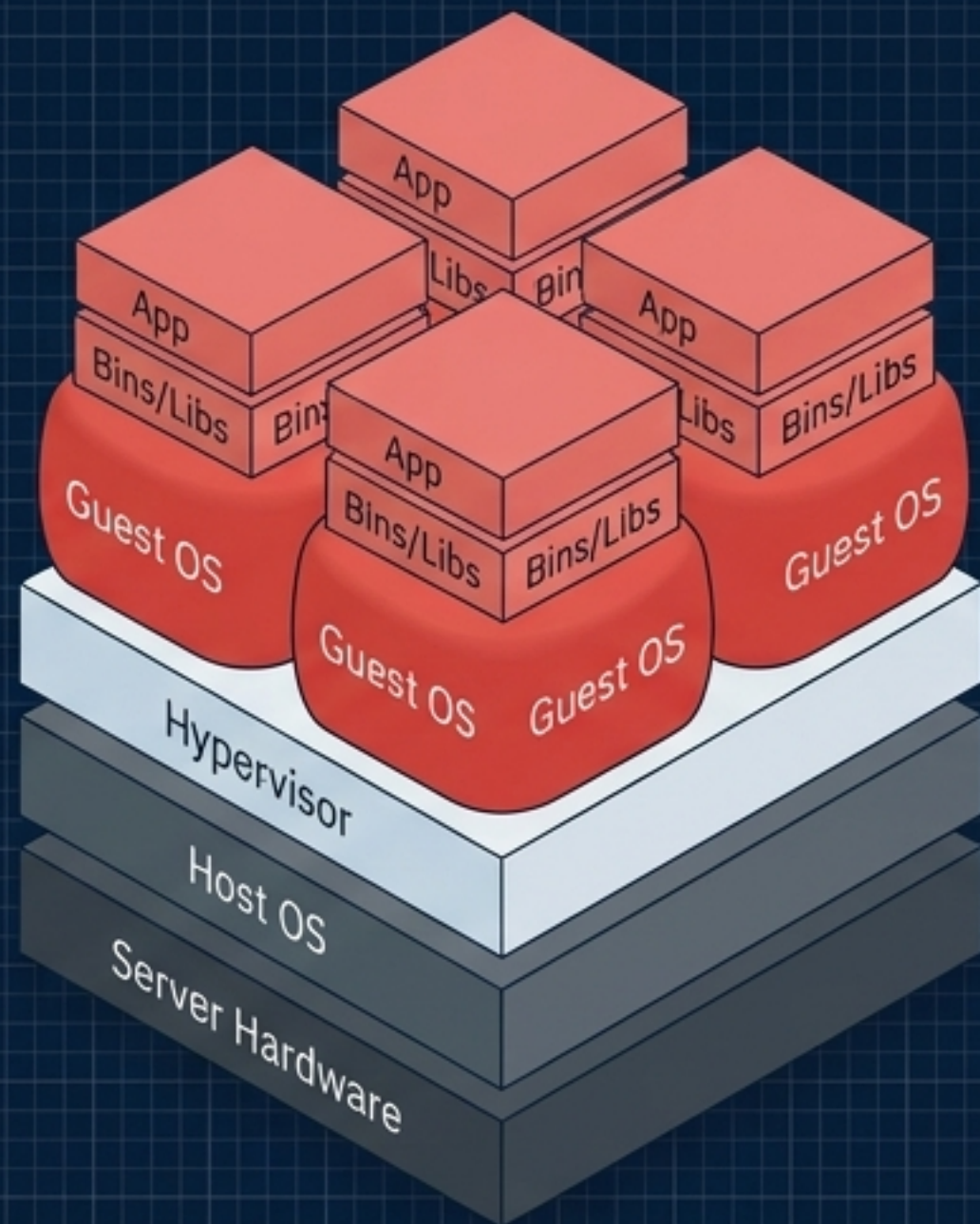
ใช้ทรัพยากรน้อยกว่าระบบเดิมอย่างมหาศาล



Consistent (สม่ำเสมอ)

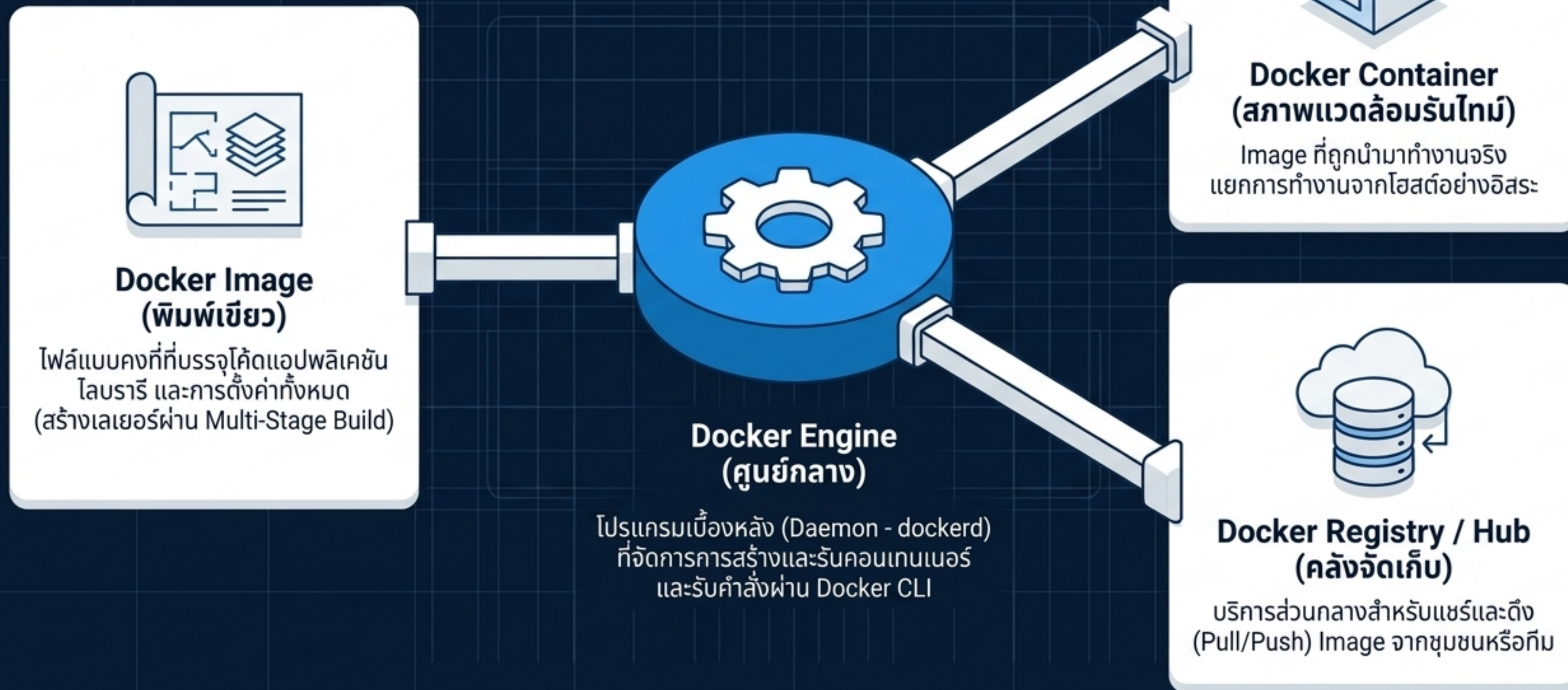
สภาพแวดล้อม Dev, Test และ Prod ตรงกัน 100%



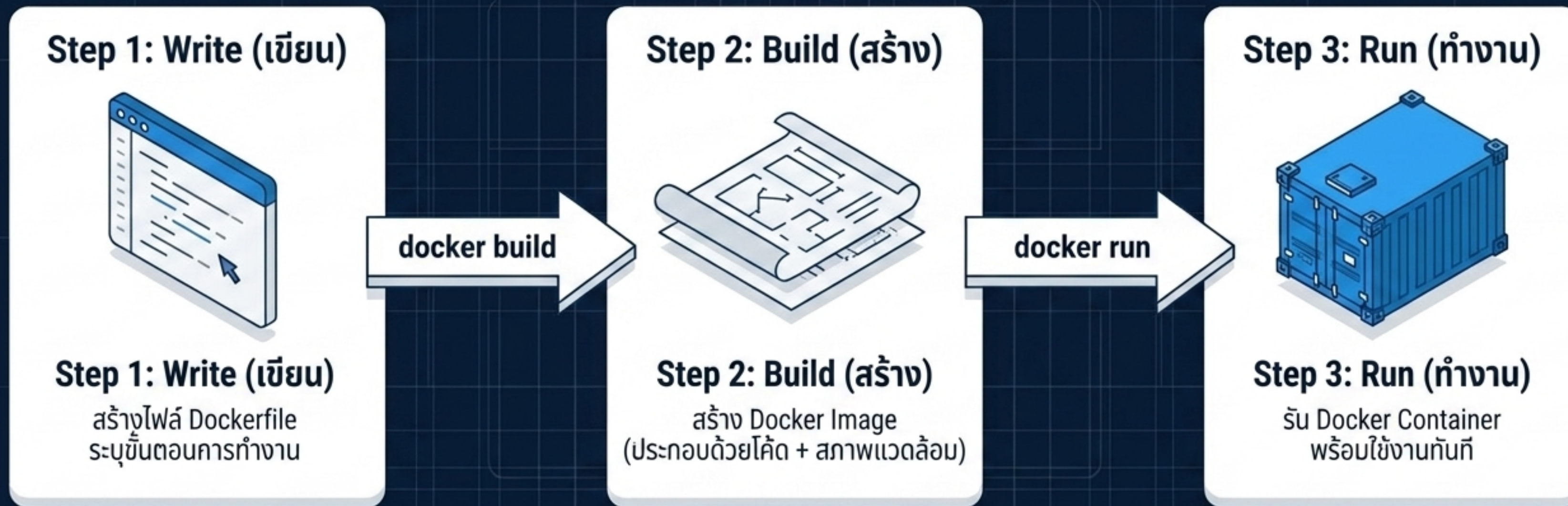


ขนาด (Size)	VM (ใหญ่/GB)	Docker (เล็ก/MB)
เวลาเริ่มต้น (Boot Time)	VM (ช้า/นาที)	Docker (เร็วมาก/วินาที)
การใช้ทรัพยากร (Resources)	VM (สูงมาก)	Docker (ต่ำ/แชร์ OS Kernel)

องค์ประกอบหลักของระบบนิเวศ Docker



Workflow ของนักพัฒนา: จากโค้ดสู่คอนเทนเนอร์



เจาะลึกโครงสร้าง Dockerfile (Node.js Example)



```
1 FROM node:lts
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7 COPY . ./
8
9 ENV PORT=8080
10
11 CMD [ "node", "index.js" ]
```

FROM: ระบุ Image ต้นแบบ
(ดึงจาก Docker Hub อัตโนมัติหากไม่มี)

WORKDIR: กำหนดโฟลเดอร์หลักในการทำงาน
(คล้ายคำสั่ง cd)

COPY / RUN: คัดลอกไฟล์และรันคำสั่งติดตั้ง
(Layering)

CMD: คำสั่งเริ่มต้นเมื่อคอนเทนเนอร์เริ่มทำงาน

เคล็ดลับ .dockerignore - อย่าลืมสร้างไฟล์ .dockerignore ระบุ node_modules เพื่อลดขนาด Image และเร่งความเร็วในการ Build!

คำสั่งพื้นฐานที่ต้องรู้ (Essential Commands)



สร้าง (Build)

```
docker build -t docker-test .
```

(สร้าง Image จาก Dockerfile
ในโฟลเดอร์ปัจจุบัน)



รัน (Execute)

```
docker run --rm -p 8080:8080  
docker-test
```

(รันคอนเทนเนอร์ และ Map Port
ของโฮสต์เข้ากับคอนเทนเนอร์)



ตรวจสอบ (Inspect)

```
docker ps -a
```

(แสดงรายชื่อและสถานะของคอนเทนเนอร์
ทั้งหมด)



จัดการ (Manage)

```
docker stop [CONTAINER ID]
```

(หยุดการทำงาน)

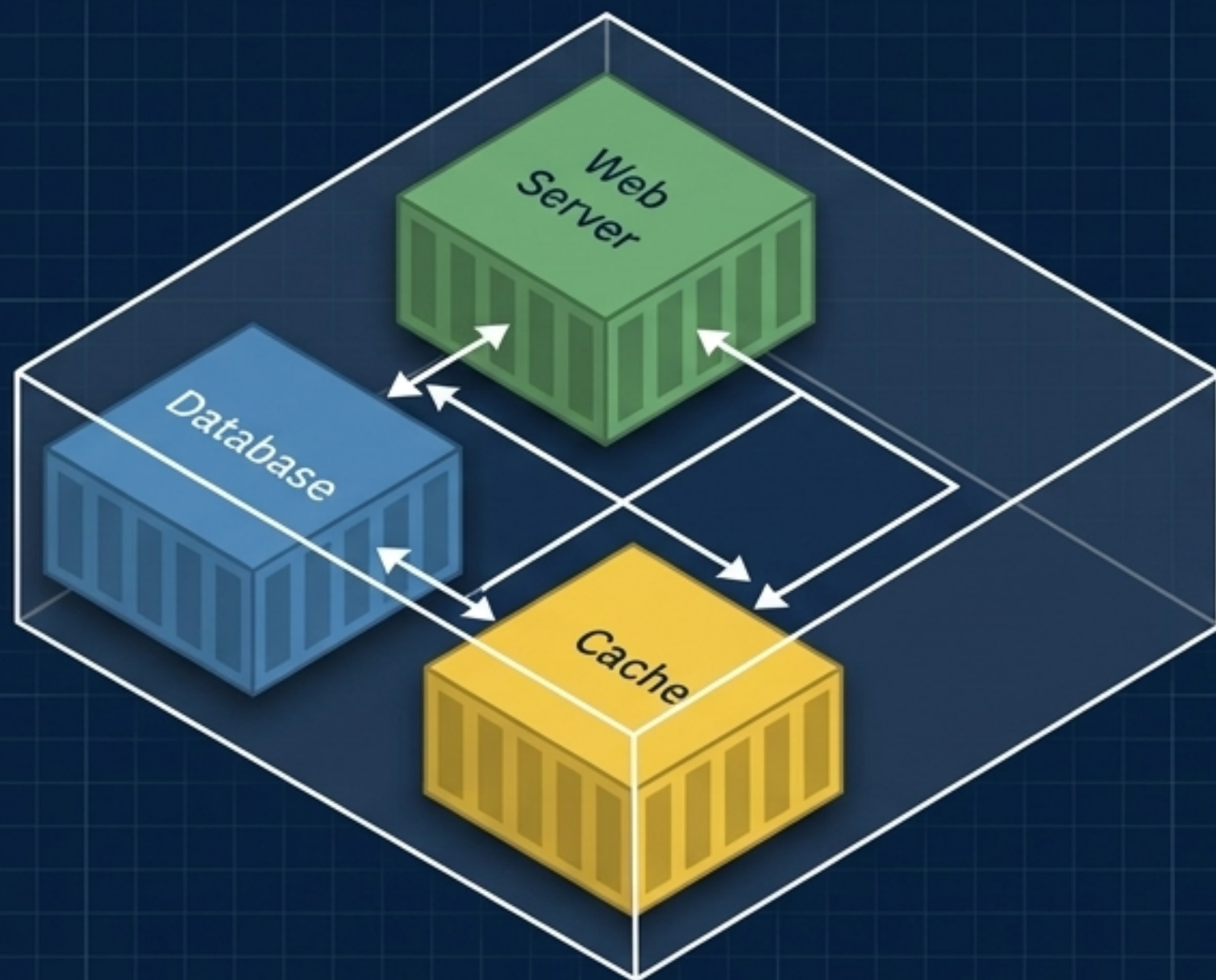
```
docker rm [CONTAINER ID]
```

(ลบคอนเทนเนอร์ทิ้ง)

ขยายสเกลระบบด้วย Docker Compose

สำหรับแอปพลิเคชันที่ซับซ้อน (เช่น Web Server + Database + Cache)

Docker Compose ช่วยจัดการหลายคอนเทนเนอร์ในครั้งเดียว



The Orchestrator

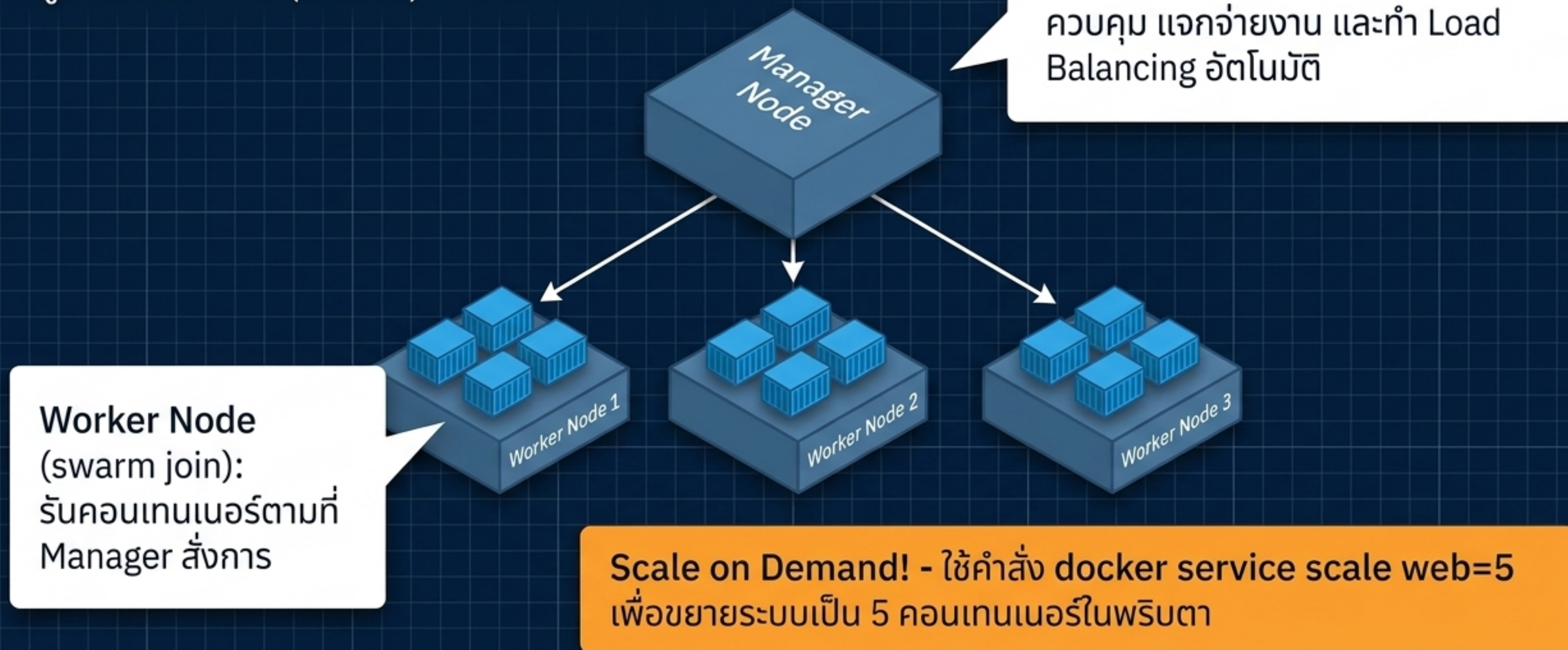
ใช้ไฟล์ `docker-compose.yml` เพื่อระบุการเชื่อมต่อเครือข่ายและทรัพยากร

One Command Magic

ใช้คำสั่ง `docker-compose up` เพื่อรันทั้งระบบ และ `docker-compose down` เพื่อหยุดทั้งหมดพร้อมกัน

สเกลระดับองค์กรด้วย Docker Swarm

จัดการคอนเทนเนอร์ข้ามหลายเซิร์ฟเวอร์ (Nodes)
ในรูปแบบคลัสเตอร์ (Cluster) เพื่อรองรับโหลดมหาศาล



Docker: หัวใจสำคัญของสถาปัตยกรรม DevOps

Microservices Backbone
แยกระบบเป็นส่วนย่อย พัฒนา
และอัปเดตอิสระจากกัน

Speed to Market
ลดความซับซ้อน ปรับใช้แอปพลิเคชันใหม่ได้ภายในไม่กี่วินาที



CI/CD Enabler
สร้างสภาพแวดล้อมที่คาดเดาได้
ทำให้การ Test และ Deploy
เป็นไปอย่างอัตโนมัติและรวดเร็ว

แนวทางปฏิบัติเพื่อความปลอดภัย (Security Best Practices)

1: Non-Root Execution

รันด้วยสิทธิ์จำกัด (เช่น `docker run -u 1000:1000`) ป้องกันการโจมตีทะลุสู่ Host OS

2: Private Networks

แยกเครือข่ายภายในเฉพาะคอนเทนเนอร์ที่ต้องสื่อสารกัน เปิดพอร์ตสู่ภายนอกเท่าที่จำเป็น

3: Verified Images

ใช้ Image จากแหล่งที่เชื่อถือได้ (Docker Official Images) บน Docker Hub เสมอ

4: Daemon Maintenance

อัปเดต Docker Daemon และแพตช์ OS อย่างสม่ำเสมอเพื่ออุดช่องโหว่



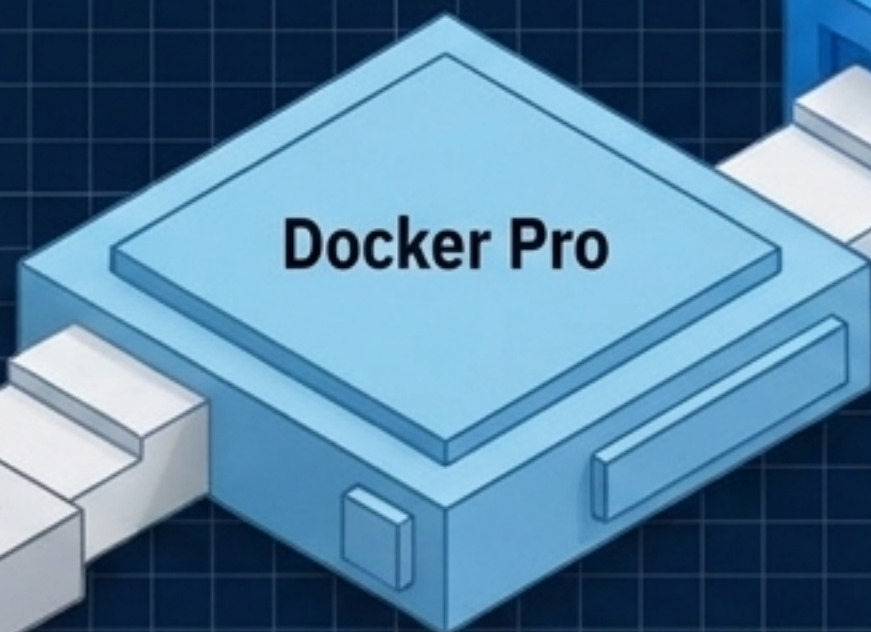
ภาพรวม: ข้อดีและข้อควรระวัง

Pros (ข้อดี)		Cons (ข้อจำกัดที่ต้องวางแผน)	
	ประหยัดทรัพยากร (ไม่จำลอง OS ทั้งระบบ)		ความซับซ้อนเมื่อสเกลใหญ่ (ต้องพึ่งพา Orchestration)
	ความเร็วสูงสุด (รันเสร็จในไม่กี่วินาที)		ความเสี่ยงด้านสิทธิ์ (หากรันด้วย Root โดยไม่ระวัง)
	ความยืดหยุ่นสูง (ทำงานข้ามแพลตฟอร์มได้ไร้รอยต่อ)		ข้อจำกัดบน Windows (ต้องใช้ Layer เพิ่มเติมเทียบกับ Linux)
	สถาปัตยกรรม Microservices เต็มรูปแบบ		

เลือก Docker Subscription ที่เหมาะสมกับคุณ



สำหรับนักพัฒนาเดี่ยว
โปรเจกต์ส่วนตัวและการเรียนรู้



สำหรับโปรแกรมเมอร์มืออาชีพ
เพิ่มโควตาดึง Image
และฟีเจอร์ความปลอดภัย



สำหรับทีมขนาดกลาง
จัดการโปรเจกต์ร่วมกัน
แชร์ Image ได้อย่าง
มีประสิทธิภาพ



ความปลอดภัยสูงสุด
จัดการทีมจำนวนมากได้ครอบคลุม
พร้อม Support เต็มรูปแบบ

AskMe Thailand: พันธมิตรทางการของ Docker

ตัวแทนจำหน่าย Docker อย่างเป็นทางการรายแรกในประเทศไทย
พร้อมพลิกโฉมองค์กรของคุณด้วยเทคโนโลยี Containerization

AskMe



Consulting

ให้คำปรึกษาและออกแบบ
สถาปัตยกรรมระบบ

Implementation

ติดตั้งและย้ายระบบ
(Migration) อย่างปลอดภัย

Support

บริการหลังการขายและดูแล
Docker Business อย่าง
ครบวงจรโดยผู้เชี่ยวชาญ

ยกระดับ IT Infrastructure ของคุณวันนี้

สอบถามเพิ่มเติม ขอคำปรึกษา หรือขอใบเสนอราคา

ช่องทางการติดต่อ

กรุงเทพฯ (HQ): 02-245-1335-7

ภาคเหนือ (เชียงใหม่): 053-858-186

ภาคอีสาน (ขอนแก่น): 096-415-4445

ภาคตะวันออก (ชลบุรี/ระยอง): 099-614-4995

ภาคใต้ (สงขลา): 094-236-9994

ช่องทางดิจิทัล



Email: sales@askme.co.th

Line: @askme (พร้อม QR Code)

Website: www.askme.co.th

